

Data Structure (week 2)

講師：陳俊豪

- DC 討論群組: <https://discord.gg/FZJJqaJd>



資料結構 (DS)

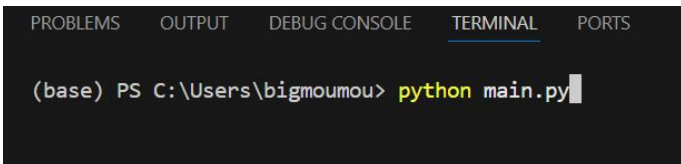
Python Environment

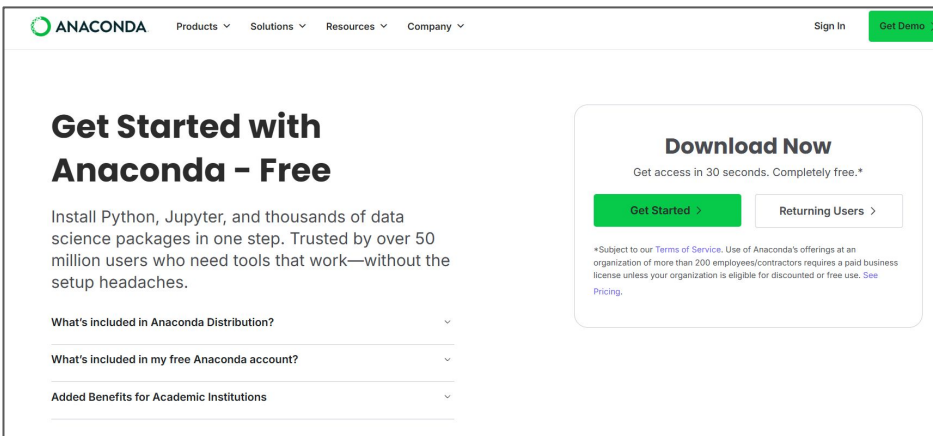
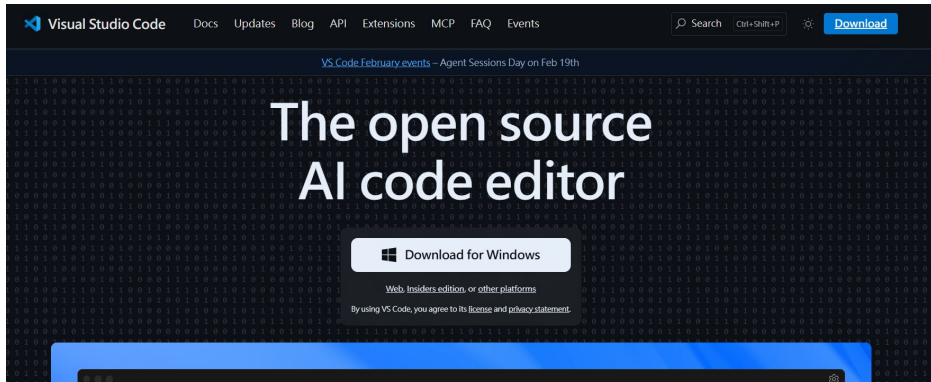
Vscode / Anaconda

Colab

VSCode + Anaconda

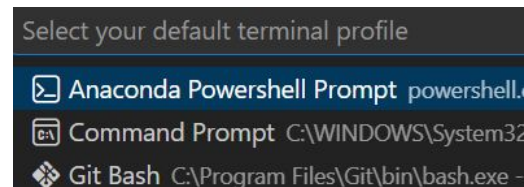
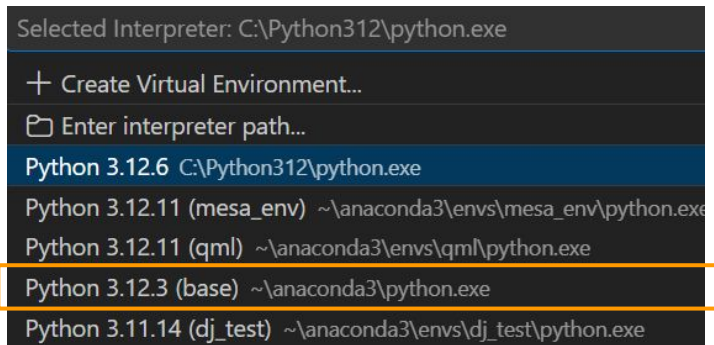
- 安裝 Anaconda 用 conda 管理環境
- 安裝 VSCode 作為編輯器 + Microsoft Python Extensions
 - 建議於下方開 terminal 用指令執行 python 檔 (Anaconda PowerShell Prompt)

- 



VSCode + Anaconda

- 安裝 Anaconda 用 conda 管理環境
- 安裝 VSCode 作為編輯器 + Microsoft Python Extensions
 - 安裝完上述後, 進到 VSCode 按下「Ctrl + Shift + P」打開指令控制板
 - 輸入並選擇「Python: Select Interpreter」
 - 在跳出的清單中, 尋找帶有 conda 或 base 字樣的 Python 路徑並選擇他
 - 再次按下「Ctrl + Shift + P」, 這次輸入並點選「Terminal: Select Default Profile」
 - 首選「Anaconda PowerShell Prompt」, 沒有才選「Command Prompt」
 - 重開 VSCode, 再打開下方終端機, 會出現 (base) 開頭就是成功



Colab - Jupyter Notebook

- Google 提供的免費 CPU / GPU 的 python 運算平台
 - 基本教學：<https://www.youtube.com/watch?v=inN8seMm7UI&t>
 - CPU 對於學習的需求是足夠的
 - GPU 免費版也足夠玩一些深度學習訓練
 - 付費版 \$10 / month
 - 運行時間: 12hrs => 24hrs
 - 可開機台: 1台 => 3台
- (還不吃自己家的電...)

Colab - Jupyter Notebook

- Theme + Miscellaneous
- Code Block
 - Python 3 / R (Change runtime type)
- Text Block
 - Markdown
- Change runtime type (Hardware)
- Free Gemini
- Share
- Connect to Google Drive

Python 基礎重點介紹

變數與型態

Python 下的 4 種變數型態 (Type)

Type 型態	中文	英文	examples
int	整數	integer	1、10、-100
float	浮點數 (小數)	floating point	-1.2、3.14、123.45
str	字串	string	'Hello'、"QQ"、"a"
bool	布林值	boolean	True、False

- 註：可用 `type()` 方法確認

單雙引都可以



注意大寫



數值運算 ([Colab 練習](#))

- Int 和 float 都可以進行 **+*/** (加減乘除) 運算。

加法

```
a = 10
```

```
b = a + 9
```

```
print(b)
```

減法

```
a = 10
```

```
b = a - 9
```

```
print(b)
```

乘法

```
a = 10
```

```
b = a * 2
```

```
print(b)
```

次方項

```
b = a ** 3
```

```
b = a ** 0.5
```

除法

```
a = 10
```

```
b = a / 5
```

```
print(b)
```

- 加減乘除 偷懶寫法:
 - `a += 1`
 - `a -= 1`
 - `a *= 1`
 - `a /= 1`

數值運算 ([Colab 練習](#))

- Int 和 float 還可以進行 `//` 和 `%` 運算。

取商數

```
a = 10
```

```
b = a // 3
```

```
print(b) # 3
```

取餘數

```
a = 11
```

```
b = a % 5
```

```
print(b) # 1
```

文字運算 (**str**) - ([Colab 練習](#))

- Int 和 float 可以進行 +-* / 運算, 那 str 也可以嗎?

文字運算 (**str**) - ([Colab 練習](#))

- Int 和 float 可以進行 $+ - * /$ 運算, 那 str 也可以嗎 ?

加法

(**paste 貼上**)

```
str1 = 'Hello '  
  
str2 = 'Adam'  
  
str3 = str1 + str2  
  
print(str3)
```

乘法

(**repeat 重覆**)

```
str1 = 'Hello '  
  
str2 = str1 * 10  
  
print(str2)  
  
print('*' * 20)
```

布林值運算 (**bool**) - ([Colab 練習](#))

- Int、float、str 都可以進行運算了, 那 bool 如何運算？

not (!)

```
bool1 = True  
  
bool2 = not bool1  
  
print(bool2) # False
```

and (&)

```
bool1 = True  
  
bool2 = True  
  
bool3 = bool1 and bool2  
  
print(bool3) # True
```

or (|)

```
bool1 = True  
  
bool2 = False  
  
bool3 = bool1 or bool2  
  
print(bool3) # True
```

--- 小括號的重要性 ---

int, float, str, bool

+ - * /

+ *

!&|==

變數命名 (**Variable Naming**) - ([Colab 練習](#))

- 大小寫有差 → Case Sensitive
 - date vs Date
- 不可以數字開頭 (why? / /)
 - 2021date
- Python 保留字不能當作變數名 → Reservation
 - if / elif / else
 - while / for
 - True / False / None
- 特殊符號不能使用 (why?)
 - \$ & @ ! #) * 但可以用

變數命名 (**Variable Naming**) - ([Colab 練習](#))

- 使用**英文**命名就好
- 英文**小寫**為主, 大寫通常有特殊情況 (命名慣例)
- 若變數名很長建議採用「駝峰式」(CamelCase) 或「底線」(snake_case) 作為分隔
 - **駝峰式**: timeStamp
 - **底線**: time_stamp
- 精簡易讀為主
 - data_collector > 直接 data 或 collector > 提示性質 data_int / data_str

Overview 一下所有保留字 (Reservation)

保留字 / 關鍵字 (keywords) 總表

補充 : None vs NaN ?

```
type(None)  
>>> NoneType
```

```
type(np.nan)  
>>> float
```



<u>False</u>	await	<u>else</u>	import	pass
<u>None</u>	<u>break</u>	except	in	raise
<u>True</u>	class	finally	is	return
and	<u>continue</u>	<u>for</u>	lambda	try
as	def	from	nonlocal	<u>while</u>
assert	<u>del</u>	global	not	with
async	<u>elif</u>	<u>if</u>	or	yield

內建函式 BIF (Build-In Function)

1. print()

```
name = 'Adam'

print(name)

age = 24

print(age)

print(name, '的年齡是', age)
```

2. input()

```
input()

input('請輸入你的名子:')

name = input('請輸入你的名子:')

print(name)
```

內建函式 BIF (Build-In Function)

3. type()

```
name = 'Adam'  
  
print(type(name))  
  
age = 24  
  
print(type(age))
```

4. 型態轉變

```
age = 24  
  
print(type(age))  
  
age = str(age)  
  
print(type(age))
```

除了 str() 還有哪些型態轉變的 BIF ?

In-class exercises 1 (用想的就好)

15 分鐘內請作答

1. 程式語言分成「編譯型」與「直譯型」，請問兩者差異在哪？其中，Python 又為哪一種程式語言？
2. 宣告變數如下，請問 var1、var2、var3 的變數 type 分別為？
 - a. `var1 = 10`
 - b. `var2 = 10.0`
 - c. `var3 = '10.0'`
3. 註解 (comment) 在程式中用途是什麼？Python 中的註解該如何實現？
4. 在 Python 中，如果想要一次註解多行，有比較快的方式嗎？
5. 我們能夠不宣告 `print()` 就使用它的原因是因為它屬於 BIF (Build-In Function)，請問常用的 BIF 中能做到型態轉換的是哪四個？

In-class exercises 2 (用想的就好)

15 分鐘內請作答

- Python 中如何快速判斷兩個東西一不一樣? 那 'Adam' 與 'adam' 在 python 中有沒有差異? 10 與 10.0 呢?
- 請問 input 功能的用途為何? 請問輸入為 10 會真的為**整數** 10 嗎?

```
data = input('Please input something :')  
print(data)
```

- Int 與 float 都能做加、減法, 那 boolean 能嗎? 為什麼?
- id() 也是一個 BIF, 請問他的功能是什麼? (可以查資料)
- 如何 print 換行, 像是以下輸出?

```
Hello  
World
```

In-class exercises 3 (用想的就好)

15 分鐘內請作答

11. 程式中 `=` 和 `==` 長的非常像, 請問兩者差在哪?
12. 請問 `111` 和 `_111` 是否都是非法變數命名, 為什麼?
13. 型態轉換中 `float` 也可以轉成 `int`, 請問轉換過後的 `int` 是四捨五入還是無條件捨去?
 - a. `ex. int(3.5)` 是變成 4 還是 3
14. 那 `round` BIF 能代表四捨五入嗎? `ex. round(1.5), round(2.5)`
15. 說出以下答案:
 - a. `10 ** 3`
 - b. `10000 ** 0.5`
 - c. `(1 == 0) ** 2`
 - d. `10 ** -1`
 - e. `5 and 1 and 0`
 - f. `not 5 or 1 or 0`
 - g. `(True or 0) and (0 or 1)`
 - h. `False or 0 and 1 or "" or "a"`

Python 基礎重點介紹

資料結構 (Container type)

Python 的四大資料結構

1. List 列表

```
name_ls = ['Adam', 'John', 'Jack']
```

```
mix_ls = ['Adam', 24, 'Jack', 30]
```

3. Dict 字典

```
name_dc = { 'Adam': 24,  
            'John': 30 }
```

2. Tuple 元祖

```
name_tp = ('Adam', 'John', 'Jack')
```

```
mix_tp = ('Adam', 24, 'Jack', 30)
```

4. Set 集合

```
demo_set = {1, 2, 3, 4, 5}
```

```
demo_set = set(1, 2, 3, 4, 5)
```



List 列表

- 代表符號：中括號 **[]**
- 儲存**有順序** (order) 的**任何變數型態**
 - list1 = [10, 20, 30, 'Adam', 99.999, True]
 - 如何表示 **順序** ? (先跳看下一頁的索引介紹)
- 四大資料結構都是 python 物件, 有提供各種方便操作的方法：
 - 添加資料
 - **append()** : 從最後添加一筆資料 ex. l1.append('Adam')
 - **insert()** : 從特定位置添加資料 ex. l1.insert(1, 'Adam')
 - **extend()** : 從最後延伸列表 ex. l1.extend(l2)
 - 移除資料
 - **pop()** / remove() ex. l1.pop() / l1.remove('Adam')

Python 中如何定義位置？Indexing 索引

List1	5	6	8	1	0	1	2	4	6
-------	---	---	---	---	---	---	---	---	---

正向索引： 0 1 2 8

反向索引 -9 -8 -7 -1

:

注意 1：位置從 0 開始計算



Python 中如何定義位置？Slicing 切片 []

正向索引

`list [ idx ]`

`list [ idx1 : idx2 ]`

`list [ idx : ]` (p.s 偷懶寫法)

`list [ : idx ]` (p.s 偷懶寫法)

反向索引

`list [ - idx ]`

`list [ - idx1 : - idx2 ]`

注意 2 : Slice 的後面那一項會 "-1" 運算後才進行索引

Q: 請問 `list1[0: 8]`、`list1[5:]`、`list1[-3:-1]` 分別是什麼？

Why 後面都要 -1 ?

- CS 中程式語言很多都是設計成「Half-open interval」也就是數學上的 $[start, end)$
 - $[start, end)$: 包含 start, 但不包含 end
- 目的：
 - 計算 length 更簡單, $list1[2:10]$ 內共有 $10 - 2 = 8$ 個元素, 避免在 loop 內的 +1 錯誤。
 - 方便切片, ex. $list1[:3]$  $list1[3:10]$, 左右兩塊的 **邊界** (3)會一樣很方便。
 - 搭配從 0 開始, 假如要拿前 5 個元素只需要 $list1[:5]$, 不用再去算 -1。

索引:	0	1	2	3	4	5
元素:	A	B	C	D	E	



內建函式 BIF (Build-In Function)

5. len()

```
list1 = [1, 2, 3, 4, 5]
print(len(list1))

list1.append(6)

print(len(list1))
```



很
常
一
起
搭
配

6. range()

很像 list 但為的是方便數值計算

range(10) # [0, 1, 2, ..., 9]

range(5, 9) # [5, 6, 7, 8]

range(0, 30, 5) # 0, 5, 10, ..., 25

從 0 開始, 減一收尾

透過 list(range(10)) 才會變成真正 list



List 內元素的賦值

- 透過 Slicing 除了能造訪 List 內元素, 還可以進行修改 (賦值 assign)
- `list1 = [10, 20, 30, 'Adam', 99.999, True]`
 - 造訪 `list1` 中的 'Adam' -> `list1[3]`
 - 接著把 'Adam' 改名為 'John' -> `list1[3] = 'John'`

[10, 20, 30, 'Adam', 99.999, True]

↑

'John'



Tuple 元祖

- 代表符號：小括號 ()
- 儲存有順序 (order) 的任何變數型態 (不可更改)
 - tuple1 = (10, 20, 30, 'Adam', 99.999, True)
 - 與 List 非常像, 主要差別在不可更動 (試著更動看看?)
- 常用操作的方法：
 - 無：)



Dict 字典 (hash)

- 代表符號：大括號 { }
- 儲存**無順序**的**任何變數型態**
 - **key: value** 成對儲存
 - dict1 = {'a': 10, 'b': 90}
 - 請問 Dict 可不可以 Slice ?
- 常用操作的方法：
 - key 可以用來索引
 - 透過 dict1['a'] 可以得到 10
 - keys()：列出所有的 key 值
 - items()：列出所有的 (key, value) 組合
- 補充 JSON 資料格式

{ 'a': 10, 10: 90 }

↑ ↑

key : value

JSON 格式

```
{
  "orders": [
    {
      "orderno": "748745375",
      "date": "June 30, 2088 1:54:23 AM",
      "trackingno": "TN0039291",
      "custid": "11045",
      "customer": [
        {
          "custid": "11045",
          "fname": "Sue",
          "lname": "Hatfield",
          "address": "1409 Silver Street",
          "city": "Ashland",
          "state": "NE",
          "zip": "68003"
        }
      ]
    }
  ]
}
```

```
[
  {
    "Id": 0,
    "FirstName": "string",
    "LastName": "string",
    "Name": "string",
    "EmailAddress": "string",
    "TerritoryId": 0
  }
]
```



Set 集合

- 代表符號：大括號 `{}`
- 儲存無順序的任何變數型態 (唯一)
 - `set1 = {10, 20, 30, 'Adam', 99.999, True}`
 - 與 List 非常像, 只是變成無序且唯一 (重覆元素自動刪除)
 - set 移除了 Slicing 的概念!
- 常用操作的方法：
 - 無：)
- 其他補充：
 - 可以與 list 互相轉換, 但要注意從 set 回到 list 順序問題
 - 常把 list 直接轉 set 已找到唯一元素

Slice



{ [(int, float, str, bool)] }

+ - * /

+ *

!&|==

Slice [a: b]

range(a, b)

從 0 開始, 減一收尾

Python 基礎重點介紹

條件與迴圈

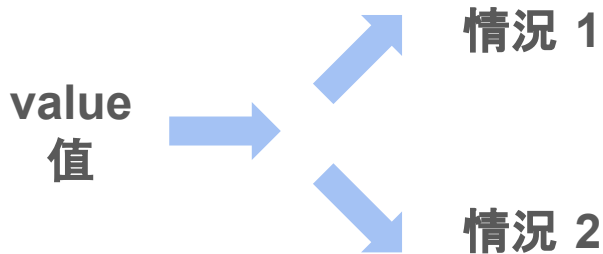
縮排 (indentation)

- Python 作者的設計美學
 - 簡化 + 乾淨
 - 前一行常搭配「:」號, 下一行「縮排」
- 常用符號: 2 個空格、4 個空格、Tab
 - 用於表示 scope (程式碼作用域)
 - 主要用於 if-else、loop、function 等語法

```
399  ∨ if __name__ == '__main__':  
400      # ===== Configurations =====  
401      → PARAMS = AttributeDict()  
402      → PARAMS.input_seq_len = 128 # 輸入序列長度  
403      → PARAMS.target_seq_len = 5  # 預測序列長度  
404      → PARAMS.runtime = 3  
405      → PARAMS.delta_t = 86400
```



單個條件 if + else



1. if + else

```
num = 77
```

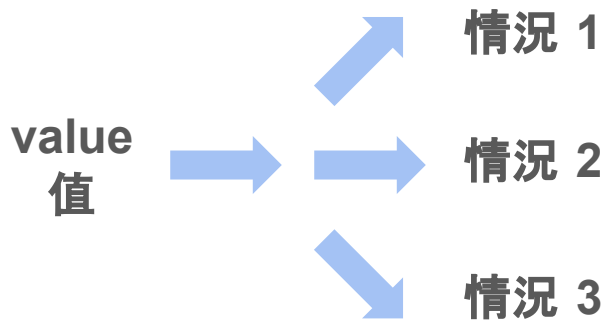
```
if num > 50:  
    print('中獎了!')
```

```
else:  
    print('太可惜了...')
```

- if / else 語法後都要加冒號
- 冒號後的下一行要縮牌
 - 什麼是縮牌？
 - 其他程式怎麼做？



多重條件 if + elif + else



2. if + elif + else

```
num = 77

if 0 < num < 50:
    print('中大獎了!')
elif 50 < num < 70:
    print('中二獎了!')
else:
    print('太可惜了...')
```

- if / elif / else 語法後都要加冒號
- 冒號後的下一行要縮牌
- (實務) 多用 elif 少用 else

最後的「in」and「not in」運算功能 (operators)

- 編寫程式中還有兩個很實用的運算功能：

in

```
list1 = [10, 20, 30, 40, 50]
```

```
ans = 10 in list1
```

```
print(ans)
```

not in

```
list1 = [10, 20, 30, 40, 50]
```

```
if 5 not in list1:
```

```
    print('列表中沒有整數5')
```



for loop 迴圈

特別設計給迴圈用的數字 List

1. 單迴圈

```
for i in range(10):  
    print(i)
```

- for 語法後都要加冒號
- 冒號後的下一行要縮牌
- BIF range() 很常搭配迴圈：
 - ex. range(10) = 0, 1, 2, ..., 9



break 和 continue

- break 和 continue 是很常搭配迴圈使用的語法。

break

中止

```
for i in range(10):
```

```
    if i >= 5:
```

```
        break
```

```
    print(i)
```

continue

跳過

```
for i in range(10):
```

```
    if i == 5:
```

```
        continue
```

```
    print(i)
```

In-class exercises 1

1. 用 for 迴圈寫出九九乘法表, 從 $1 \times 1 = 1$ 到 $9 \times 9 = 81$, 結果需要如下圖:

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
2 x 1 = 2
2 x 2 = 4
```

...

```
8 x 9 = 72
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
```

In-class exercises 2

```
ls1 = [1, 2, 3, 4]
for i in range(len(ls1)):
    p = ls1.pop()
    ls1.append(p)
print(ls1)
```

>>> Ans ?

>>>

```
ls1 = 'admin'
for char in ls1:
    if char == 'm':
        continue
    print(char)
```

>>> Ans ?

>>>

```
var = 10
while var:
    var -= 1
    if var > 5:
        continue
    print(var)
```

>>> Ans ?

>>>



In-class exercises 3

1. 給定 `list1 = [10, 20, [300, 400, [5000, 6000], 500], 30, 40]`, 請問如何使用索引 (slice) 拿到整數 6000 ?
2. 寫一個使用 `if-elif-else` 與 `input` 的程式, 程式功能要能夠判斷 `input` 的值是不是 5 的倍數, 如果是就 `print 'Yes!'`, 否則就 `'No!'`。

參考 `data = input('Please input something :')`

In-class exercises 4

```
{ [ ( int, float, str, bool ) ] }
```

 $+, -, *, /$

+ *****

1. 寫一個程式實現以下結果(結果一樣就全對):



← 8個*

2. 寫一個程式實現右方結果(結果一樣就全對):

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```


For 迴圈最後一哩路：enumerate

- Enumerate 出現的目的是為了讓迭代更為方便，更為 pythonic！
- 哪裡比較方便？

```
x = [10, 20, 30]
y = [1, 0, 1]
for i in range(len(x)):
    data = x[i]
    label = y[i]
```

```
x = [10, 20, 30]
y = [1, 0, 1]
for i, d in enumerate(x):
    data = x[i]
    label = y[i]
```

- 隱藏變數 (垃圾機制) 技巧，並不是真的消失，只是一種公約跟暗示。
 - for _, d in enumerate(x)

while 迴圈

1. 無窮迴圈

```
while True:
```

```
    print('Hello!')
```

- while 語法後都要加冒號
- 冒號後的下一行要縮排
- 如果不設中止條件 while 迴圈就不會中止。
- 通常不確定要迭代幾次就會使用 while 迴圈。

while 迴圈

2. 中止條件

```
num = 3  
  
while num:  
  
    print('Hello!')  
    num -= 1
```

- while 語法後都要加冒號
- 冒號後的下一行要縮牌
- 如果不設中止條件 while 迴圈
就不會中止。
- 左方為什麼會中止？

In-class exercises 5

1. 請把下圖被塗掉的部分補上, 以得到右方的結果。

```
a = 4
while a < 8:
    print(a)
```

4
5
6
7

2. 請把下圖被塗掉的部分補上, 以得到右方的結果。

```
ls1 = []
while len(ls1) < 4:
    num = int(input('請輸入一個數字: '))
    if num < 2:
        ls1.append(num)
print(ls1)
```

請輸入一個數字: 10

請輸入一個數字: 3

請輸入一個數字: 2

請輸入一個數字: 2

請輸入一個數字: 2

[10, 2, 2, 2]

挑戰 1：設計簡易登入系統 (terminal 實現)

```
請選擇操作選項 (a 註冊, b 登入, c 退出) ? a
```

```
請輸入帳號 : aaa
```

```
請輸入密碼 : 111
```

```
註冊成功!
```

```
請選擇操作選項 (a 註冊, b 登入, c 退出) ? a
```

```
請輸入帳號 : aaa
```

```
帳號已存在，請重新輸入!
```

Debug 需要注意的事項

- `SyntaxError: EOL while scanning string literal`
 - 打錯字、括號沒刮到、在判斷式當中 `==` 誤寫成 `=`
- `IndentationError: expected an indented block`
 - 縮排沒縮好
- `TypeError`
 - `'1' + 1`
- `IndexError: list index out of range`
 - `ls1 = [1, 2, 3]`
 - `ls1[3]`
- `ZeroDivisionError: division by zero`
 - `1 / 0`

Debug 需要注意的事項

- 可以使用 try except 去避免錯誤發生, 但一開始不建議

try:

要執行的程式碼

except 例外名稱:

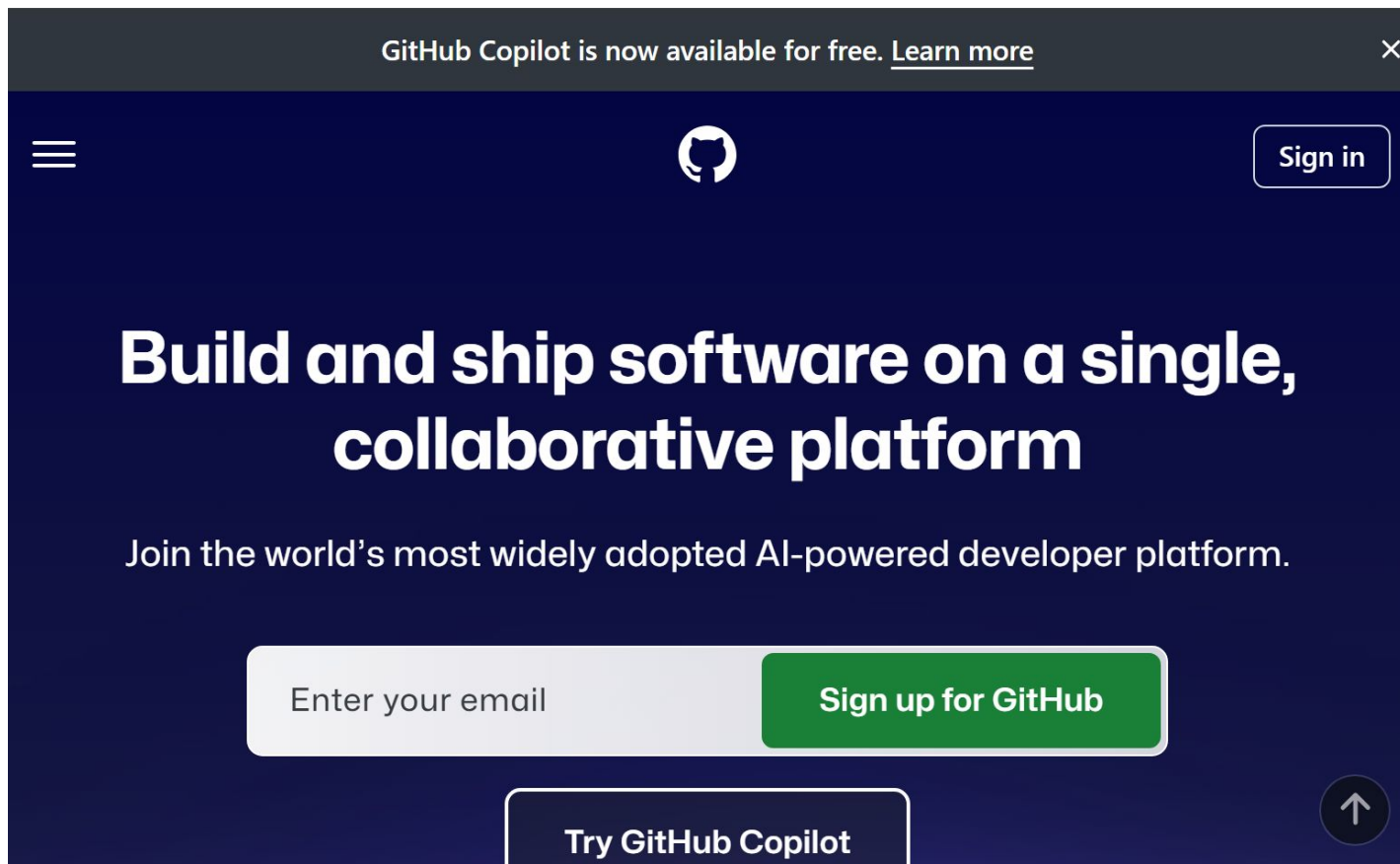
當遇到例外時要執行的程式碼

finally:

不管怎樣都必須執行的部分

```
try:
    a = 1
    b = 0
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by 0! Change value b")
    b = 0.0000000001
    print(a / b)
```

補充：[GitHub](https://github.com)



補充資訊

AI Tools 建議 (目前沒有 gemini 學生方案)

- Google Gemini 組團最便宜 (家庭群組)

- 家庭群組上限 6 人
 - 每月 NT 650 / 6 = NT 108
- (optional) 新帳號再省一個月

- 重要功能

- (Web) Gemini 3.1 Pro Model
- NotebookLM
- (Local Coding AI) Antigravity IDE
- Google One (共用) 空間

Google AI Pro

~~\$650~~ \$0/月

為期 1 個月，之後每月 \$650

取得 Google AI Pro

解鎖強大新功能的更高存取權，激發創意並提升工作效率

◆ Gemini 應用程式

獲得更高存取權，使用我們最強大的 3.1 Pro 模型、3.1 Pro 的 Deep Research，並解鎖部分 Veo 3.1 影片生成功能



Flow

取得更高的 AI 電影製作工具權限，可運用 Veo 3.1 創作電影風格場景和故事



Whisk

使用 Veo 3 圖片轉影片生成工具，可享更高用量



每月 1,000 點 AI 點數

適用於 Flow 和 Whisk



Google 搜尋

享有更高權限，可使用 Gemini 3 Pro 模型等功能



NotebookLM

享更高權限，可使用語音和影片摘要、測驗等功能，與我們的研究夥伴互動



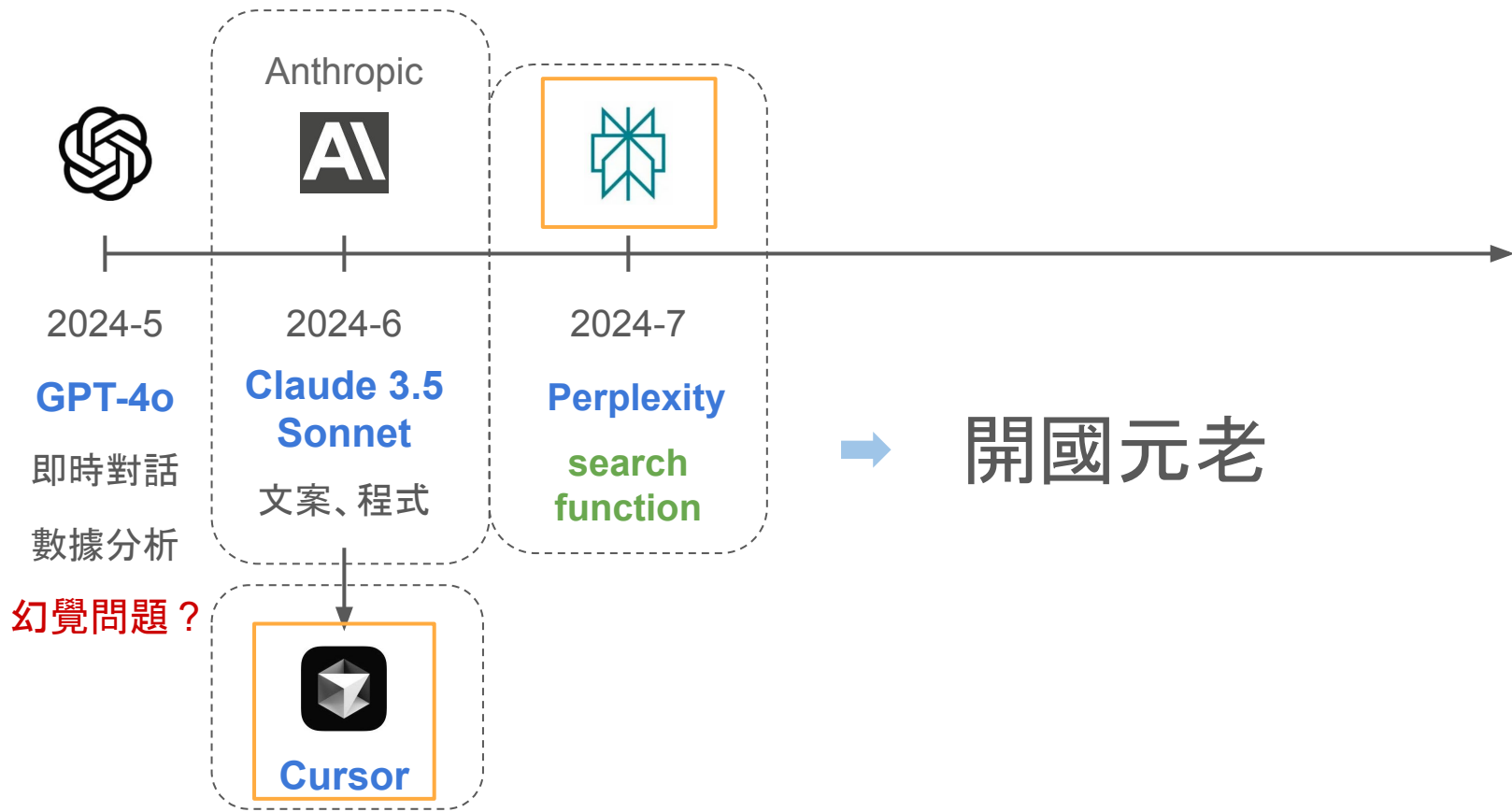
Gmail、Google 文件和 Vids 等服務內建 Gemini 直接在 Google 應用程式使用 Gemini



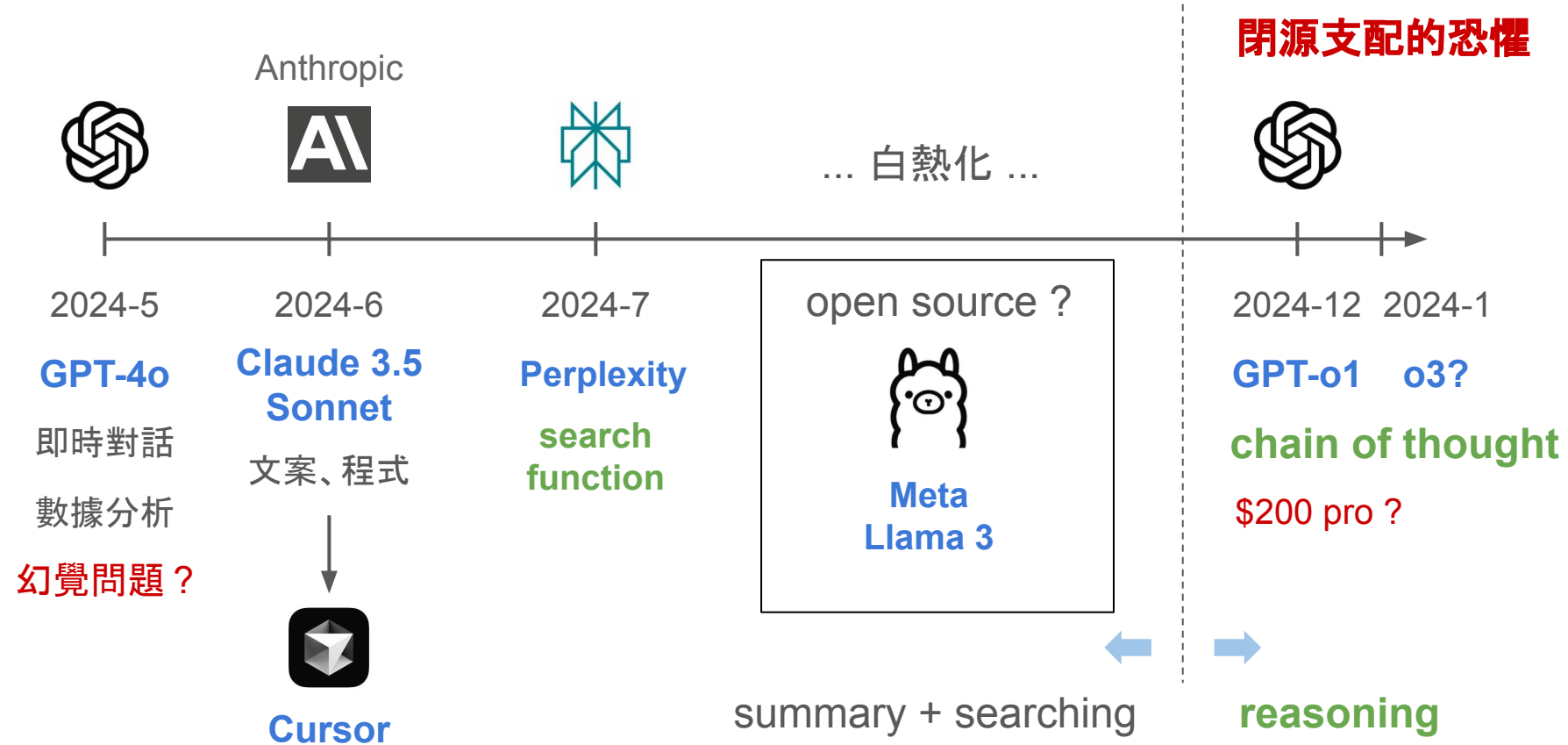
Google Antigravity

Google Antigravity 是一款代理開發平台，支援更高的代理模型存取速率額度

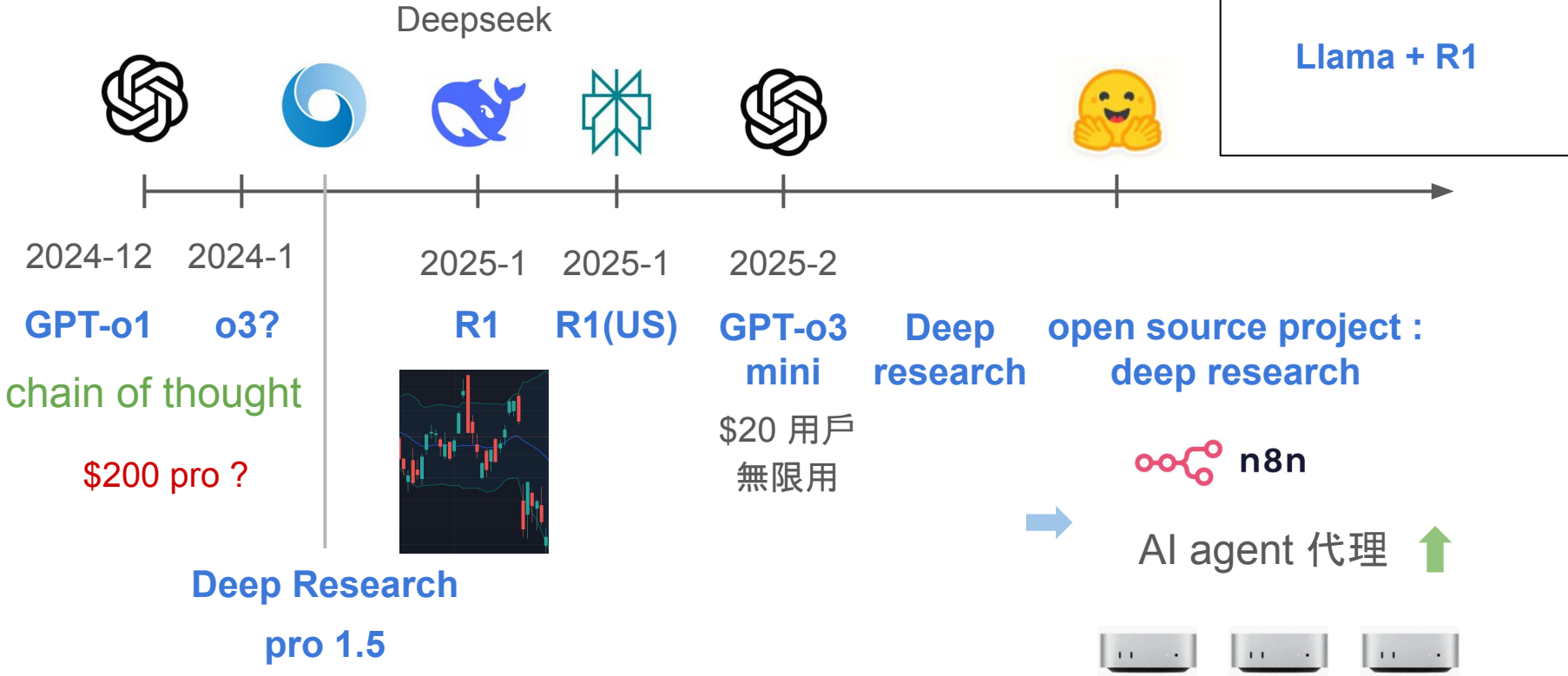
General AI tools - the new era



General AI tools - the new era



General AI tools - the new era



General AI tools - the new era

